The Publishing Accelerator Toolkit (PacTk)

Amy Neustein, Ph.D.
CEO and Founder
Linguistic Technology Systems
Fort Lee, NJ
917-817-2184
amy.neustein@verizon.net

Class Libraries and Full-Text Search/Encoding Tools for Publishing Documents, Datasets, and Multimedia Resources

Overview

In contemporary publishing, the standard workflow includes XML-based formats such as JATS or BITS, from which are compiled PDFs and other human-visible documents. Intermediate XML and similar files are distinct from those that authors submit directly; they have their own tool stacks and are computationally accessed via protocols such as SAX (Simple API for XML), XQuery, DOM (Document Object Model), and XQSE (XQuery scripting extension). Unfortunately, XML itself is not explicitly built for representing textual data, and examining JATS or BITS files via generic XML technology renders certain publishing-related query and editing tasks difficult to implement. The PDF format is similarly opaque because internal PDF representations of document text include many anomalies due to ligatures, hyphenation, font specs, and other layout-related discrepancies between XML inputs and user-visible output. As a result, internal PDF text only partially corresponds to textual content as understood by authors, editors, and human readers.

Given the limitations of PDF and XML for natural-language documents, numerous competing formats have been developed, providing potential alternatives or extensions to XML. These include TEI (Text Encoding Initiative), TagML (Text-as-Graph Markup Language), BioC (a biomedical text mining standard defined by PubMed Central), and Research Object Bundles (a Semantic-Web based specification that covers both text and data sets). With a proliferation of different text-encoding standards, query and traversal protocols that are developed for one format (e.g., JATS and its derivatives) cannot be directly applied to others. For example, the XML parent-child element relationship is replaced in TagML with two different containment styles (logical and extensional), such that DOM and event-based parsers targeted at the simpler XML hierarchy must be reimplemented for the TagML system. Meanwhile, with regard to PDF, effective search and text-interaction capabilities depend on supplementing this format's internal natural-language representation with machine-readable text encoding, which might be embedded in PDF files or available as sibling assets. In principle, XML documents could supply such content, although current pipelines are not set up to share JATS/BITS files (for example) in end-user contexts.

From the perspective of PacTk, text encoding evinces a spectrum of distinct *topomorphic regimes* where a proliferation of incompatible document structures prohibits the emergence of common query, traversal, and metadata formats. The obvious problem is that workflows have to be reconstructed for each incompatible document specification. In practice, such difficulties have almost certainly hindered the adoption of theoretically well-founded formats such as TagML.

The idea behind PacTk is that divergent formats cannot be satisfactorily reconciled simply by proposing yet another format to insert into analogous workflows. Instead, building a more sophisticated publishing platform involves constructing integrated computational tableaux weaving together text-encoding, query-evaluation, compiler-theoretic, and front-end concerns.

One important variation amongst text-encoding formats is the level of granularity for textual elements. In JATS, BITS, and most JSON encodings, the fundamental discursive unit is the individual paragraph. Elements on smaller scales (such as sentences or block quotes) are notated haphazardly at best. LaTeX employs reasonable sentence-boundary heuristics — which may be overridden via macros when necessary — but LaTeX sentences are not expressed in any systematic manner (e.g., via auxiliary files) for any purposes other than visual spacing. In TEI and BioC, individual sentences *can* be represented, but this is not mandatory (thus **s** and **sentence** tags are available but not guaranteed to be present in any particular document). By contrast, PacTk is explicitly built around sentence-level units, specifically a "topomorphic sentence-level object model" (TSOM): when all regular text in a natural-language document is explicitly contextualized in a delineated sentence object, many editing, indexing, and user-interface tasks become feasible (as outlined in the next section).

In contrast to the other formats mentioned here, PacTk is not a single file type but rather a collection of code libraries and classes, with data types representing specific textual elements (sentences, paragraphs, PDF pages, annotations and comment-boxes, etc.). Query and traversal protocols modeled via XQuery, DOM, and so on become expressed by object methods and operators exposed to C++ code, for example (PacTk is thus "topomorphic" in the sense that navigation between objects is determined by structural properties of divergent document formats, that manifest in competing notions of individuation for structure-sites and any interrelationships declared between them). In short, the PacTk object system synthesizes distinct structuring protocols found in competing formats such as JATS, TagML, and BioC, allowing publishers' workflows to encompass each of these formats (and others) without sacrificing compatibility. PacTk is particularly aimed at publishers who feel constrained by the limitations of JATS or BITS but are reluctant to try alternatives (TagML, BioC, etc.) due to minimal software-ecosystem support and the lack of standardized XQuery and DOM-style interfaces.

PacTk's design also reflects trends in scientific research and the emergence of interdisciplinary fields such as social ecology, translational medicine, and medical humanities. It is the viewpoint of **PacTk**'s developers that the value of new investigations cannot be fully expressed in single-disciplinary terms, nor does one individual publication mark the end of a well-conceived research project. Instead, publications are milestones in the unfolding of projects that, in the best case, progress to socially-beneficial deployments. More so than in the past, research projects (summarized but not fully articulated by academic papers) can serve as foundations for scholars' and scientists' technical "identity," potentially more significant than details such as degree specialization or disciplinary affiliation. In recognition of these paradigms, **PacTk** seeks to help authors build multi-faceted presentations covering data, code, and foundations for practical applications, which authors could build upon for future research work, funding, translational implementation, and implementational feedback.

The remainder of this document will discuss how **PacTk** can streamline editing, indexing, and document-preparation tasks, as well as improve User Experience for human readers. Later sections will also address how **PacTk** may contribute to broader projects where text documents are published alongside data sets and/or multimedia presentations. Overall, **PacTk** presents a unified model and coding interface for text queries operating in diverse contexts, including **PDF** viewers, search engines, document-preparation code, and editing/typedetting. Similarly, software employing **PacTk** to navigate through document objects can leverage interop metadata across multiple formats — such as **XML** elements and **PDF** page coordinates — that is typically unavailable in other publishing tools.

Features for Editing, Indexing, and PDF Viewers ▸▸▸

The computational foundation of **PacTk** is a collection of classes that represented individual sentences and other textual elements in the **TSOM** family (**PDF** pages, annotations, footnotes, block quotes, edits or changes across document versions, etc.). In order to initialize these objects, **PacTk** provides a distinct input format called **GTagML**, or "grounded" **TAGML**. The term "grounded" reflects how certain structure-sites may be specifically marked as serializing a typed object: i.e., tags, attributes, or text nodes in the neighborhood of a site supply the information necessary to initialize an object-instance during deserialization. In other respects **GTagML** follows most of the principles advocated by **TAGML**, with novel extensions; for example, **GTagML** encourages the exploitation of Unicode Private Use areas to disambiguate visibly similar but logically distinct characters/glyphs (e.g., periods may embody punctuation, acronyms, or abbreviations).

In source files, **GTagML** is loosely based on Markdown, so authors familiar with Markdown from more informal contexts (e.g., social media, chat rooms, **GIT** documentation, etc.) will recognize many of the **GTagML** styling commands. **GTagML** sources might be provided by authors directly, or derived from author inputs by digital copy editors or others on the manuscript pipeline. To support **GTagML**, **PacTk** provides a new **C++** parsing library that is readily extensible (more so than parsers for **XML**, **JSON**, and so forth), allowing publishers to customize the **GTagML** format with new tags, syntax, and data structures specific to their in-house styles and disciplinary focus areas (e.g., specialized tags/data for different scientific fields — chemistry, physics, medicine, etc.; or whatever subject areas are addressed by given publishers, journals, and book series). Alternatively, **TSOM** objects could be compiled from sources such as **JATS** or **BITS** directly.

For workflows that feature **JATS**, **BITS**, or LaTeX, **TSOM** objects may be employed as adjunct data sources through which edits, queries, document version-comparisons, and other computational artifacts are routed. **PacTk** data can then be mapped both to archiving formats (like **XML**) and camera-ready generators (like LaTeX), but with a common **TSOM** background these two different representational facets may be integrated (for example, **XML** elements mapped to corresponding **PDF** page coordinates). In effect, **PacTk** serves to triangulate human-visible presentations (like **PDF**) with behind-the-scenes **JATS**-style formats. Analyses that are currently implemented against **XML** structures may be more efficiently coded via **PacTk** objects, with concrete benefits in areas such as copy-editing and indexing.

▷ PacTk for Editing

PacTk has an advanced query framework to help find specific locations in text where edits might be warranted (e.g., to comport with journal or book-series conventions) and to correctly review and process such edits. Here are some features:

Contextual Filters for Text Locations Often it is useful to restrict the scope of text searches to content with specific presentation characteristics (italics, alternative typefaces, citations), discursive roles (inline/block quotes, footnote text, section/subsection titles), or semantic significance (acronyms, annotations, proper/place names).

Implementing such filters via **XPATH**/tag restrictions can be inaccurate, because relevant filter criteria could be orthogonal to **DOM** hierarchies. For example, **XML** or LaTeX code might wish to wrap elements such as footnotes or block quotes in custom environments, to ensure desired pre- or post-processing. Material that is functionally footnote or quoted text, therefore, may not necessarily be indicated with those specific tag names. The **PacTk** system allows programmers to define lists of boolean parameters that apply to text-sequences and then activate those flags automatically in specific tag contexts.

In general, **PacTk** supports queries where ordinary textual restrictions may be augmented with filters for a wide variety of layout and discursive contexts. Such queries could then be used for editing tasks that should be localized to specific contexts, or where a user wants to find a specific piece of text known to have a given context.

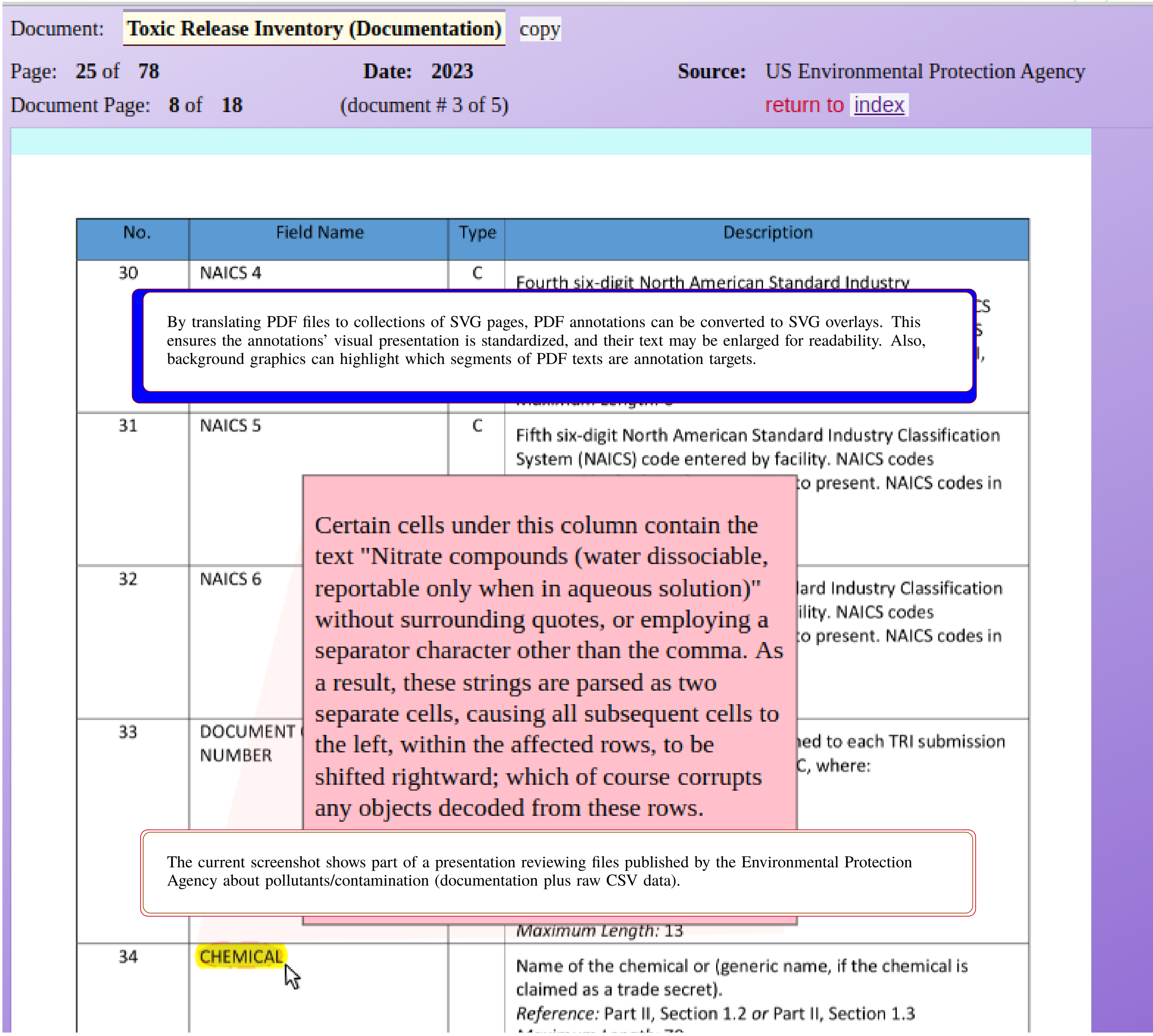


Figure 1: Mapping PDF pages to SVG views for improved annotation support

Sentence-Level Queries Siting queries relative to sentence boundaries can improve precision and express query rationales more succinctly (when compared to paragraph-based organization). For instance, the proper punctuation around a word or phrase often depends on whether that content appears at the beginning, middle, or end of a sentence. Some query expressions might similarly depend on stipulating that two or more search targets (e.g., two proper names) occur in the same sentence.

Because **PacTk** recognizes almost all text as contained inside sentence objects, queries dependent on sentence boundaries can be directly evaluated on TSOM data (such boundaries are unambiguously notated via input sources, rather than detected approximately using Natural Language Processing). **PacTk** likewise models "topomorphic" navigational relations between sentences, such as between adjacent siblings of parent objects (typically paragraphs) and also nestings wherein a multi-sentence footnote, for example, occurs in a parent-sentence context, or where a block quotes functions in part as a logical child of the sentence that introduces it.

PDF Page Coordinates Human readers post-publication — plus authors and copy editors beforehand — typically view documents in PDF format for normal reading, even if edits/changes are made on intermediary files. As a result, a bitmapped page-image is the primary medium through which users view single publication pages. Such graphics are typically rendered by PDF viewers, but other techniques might be used, such as converting page images to SVG (illustrated here in Figure 1, where the relevant component is an embedded web engine that communicates with a PDF viewer by **QWebChannel**; so SVG events are processed by C++ code directly, having only a minimal JavaScript routing layer). Failure to interoperate these images with structured text formats is a significant hindrance to efficient document preparation and editing.

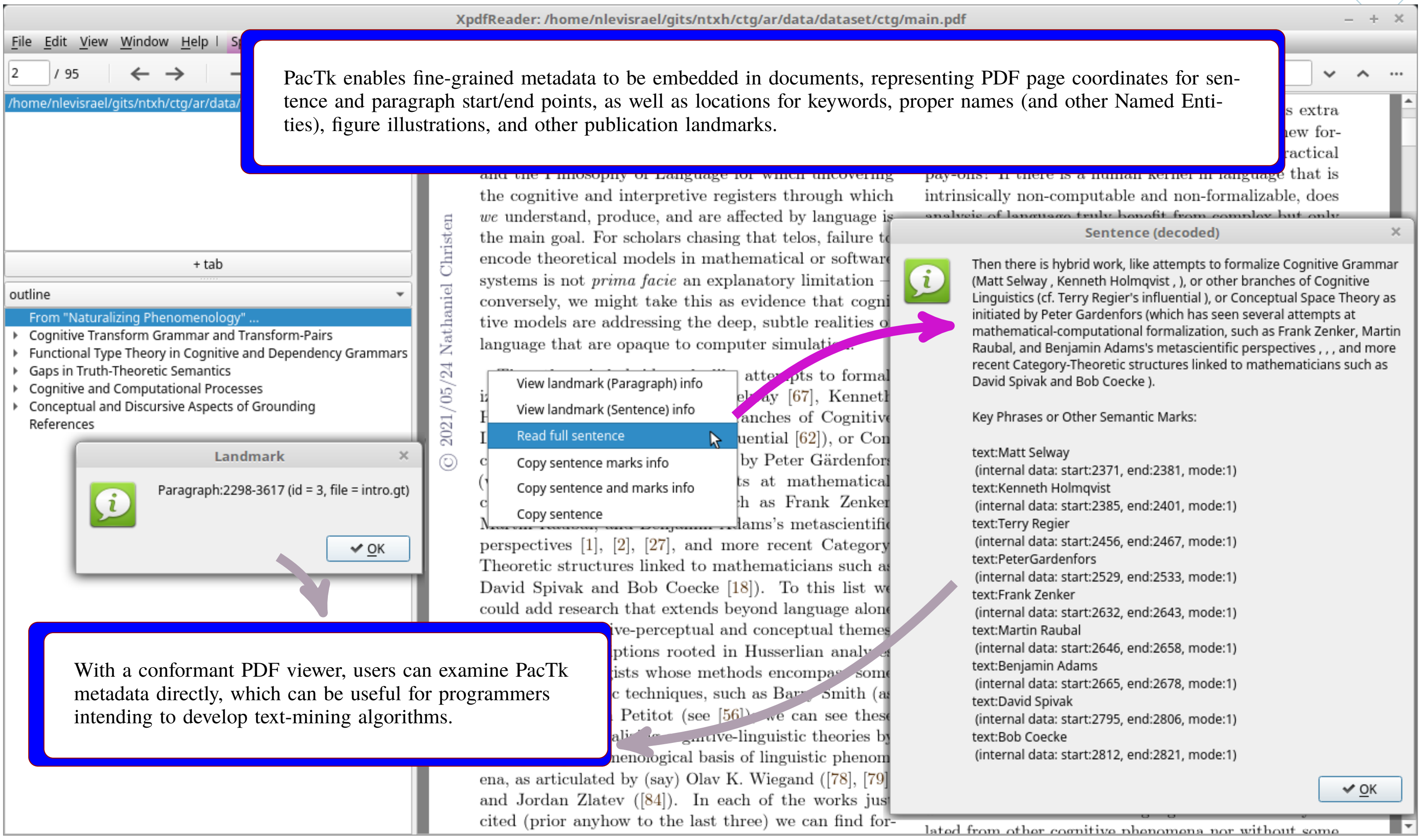


Figure 2: Enhanced GUI capabilities enabled by machine-readable text encoding

By design, **PacTk** supports post-processing algorithms within pipelines whose formats can generate **PDF**-coordinate metadata, such as LaTeX **pgfsavepos** (and auxiliary **write**) commands. Page coordinates for significant discursive locations (e.g., sentence and paragraph boundaries, footnote markers, keyphrases and named entities, citations) can thereby be exposed to **PDF** viewers which could accordingly implement novel front-end features such as overlays, context menus, and dialog windows positioned nearby relevant content (Figure 2). For example, a context menu action may include copying the full text of a current sentence to clipboard, based on the screen position where the menu is activated; semitransparent overlays could similarly show the extent of individual sentences during mouseover events.

PDF Comments and Annotations In addition to sentence objects, **PacTk** classes model data specifically relevant to the **PDF** front-end milieu, including single pages, links/anchors, and comments/annotations. Objects such as comment-boxes may thereby be leveraged systematically in an editing/review pipeline.

In particular, **PacTk** supports construction of hashtag- or handle-style controlled vocabularies that could annotate **PDF** comments introduced by copy editors or typesetters. Many modifications considered during the editing process follow common patterns, such as proper use of commas around subordinate clauses, or correct formatting of ellipses and bracketed content within inline/block quotes. Assuming that editors consistently include notations for recurring patterns, **PacTk** code can filter queries specifically to **PDF** comment boxes which indicate a specific editing category. Index codes within **PDF** comments may likewise indicate which individual sentence surrounds the annotation, so that the **PDF** view could be cross-referenced against input sources, which facilitates review and processing of manuscript changes.

For similar reasons, **PacTk** supports abstractions such as **XML** custom character entities and LaTeX macros. Many familiar textual components — ellipses, *m*- and *n*-dashes, quotation and footnote marks, etc. — are typically represented not via their visible characters but rather through standardized codes or glyphs. The problem is that systematic mappings are lacking when mixing different file types (e.g., LaTeX and **XML**, such as **JATS** or **BITS**) in a single workflow. **PacTk** has a flexible abstraction system that allows character and macro codes to propagate across different generated files.

A related problem is that abstraction codes should generally be opaque to human users in contexts such as comment boxes and copy-paste between software applications. **PacTk** therefore supports declarations for how custom entities/macros and Unicode Private Use Plane (or area) codepoints should be rendered in plain-text contexts as well as structured formats such as **XML** and LaTeX.

Often users may prefer to initiate or review edits through **PDF** files directly, but in other cases plain-text files might be appropriate instead, without **GUI** overhead. With **PacTk**, for example, authors could receive a pair of text files (or collections of files indexed, for instance, by chapter) summarizing changes between two manuscript versions. Via sentence-level filtering, such files could be organized such that only sentences that differ from one version to the next are included, with each sentence listed separately. This may be a more convenient setup for authors to review changes than working through typical **GUI**s (structured text encoding is important because it ensures that version comparisons show valid differences, rather than anomalies due to inconsistent characters and symbolization). Of course, such breakdown is only possible when the text encoding systematically identifies individual sentences, as with **PacTk**'s **TSOM** object model.

▷ PacTk for Indexing and Search Engines

Authors might use one of several methods to compile an index. They can start with a list of important headings (names, concepts, etc.) and search for relevant passages in the manuscript. Alternatively, authors could read through a document and identify when a particular paragraph mentions a named entity suitable for indexing. Depending on representations or tag sets used (e.g., the **JATS** **string-name** element) annotated spans and citations may also form the provisional heading-list for an index.

Given these various index-preparation methods, it is useful to extend front-end tools with capabilities specific to indexing workflows, such as query matching and manipulating index entry objects. Structured formats like **JATS** do not provide sufficient computational tractability in themselves. For example, the most popular desktop **GUI** framework are the **Qt C++** libraries, which are the basis of cross-platform **PDF** viewers such as Poppler and **XPDF**. **PacTk** can seamlessly interoperate with **Qt**, exposing sentence texts as **QString** objects and expressing **GTagML** parsing grammars as **QRegularExpression** instances, paired with lambda callbacks activated by the first matching rule (relative to the current parse-position). More precisely, **GTagML** has context-sensitive grammars that can include or skip over rule-tests via activated contexts; this allows sections of a **GTagML** file to utilize syntax different from the default.

With respect to indexing, **GTagML** has its own notation for index entries (headings, locators, and "see also" style redirections), to serialize indexes in a human-readable format. Correspondingly, **PacTk** has classes specifically for index entries and associated data. These objects could be initialized from **GTagML** files or specialized windows floating atop **PDF** viewers.

Building indexes on the basis of native sentence and index-entry objects has certain benefits as compared to generic (e.g., **XML**) manuscript formats. These include:

Multiple Match Strategies Index locators are typically identified by matching text against entry-headings, but the parameters of these searches will vary by context. Sometimes locators are only relevant when the local sentence includes just the exact heading label, but in other places a desired match could include words in different order (e.g., either first-name/last-name or vice-versa), or a subset (e.g., last name only), or associated concepts with different language entirely (e.g., "The White House" in lieu of a president's name). Concepts can also be grouped into semantic "clusters" (consider *ranson*, *extortion*, *bribery*, *corruption*, *influence peddling*, *lobbying*), with internal similarity-weight matrices. In the presence of a particular index entry, front-end displays can give users options to fine-tune which match strategies are appropriate for query strings seeking locators specific to that entry.

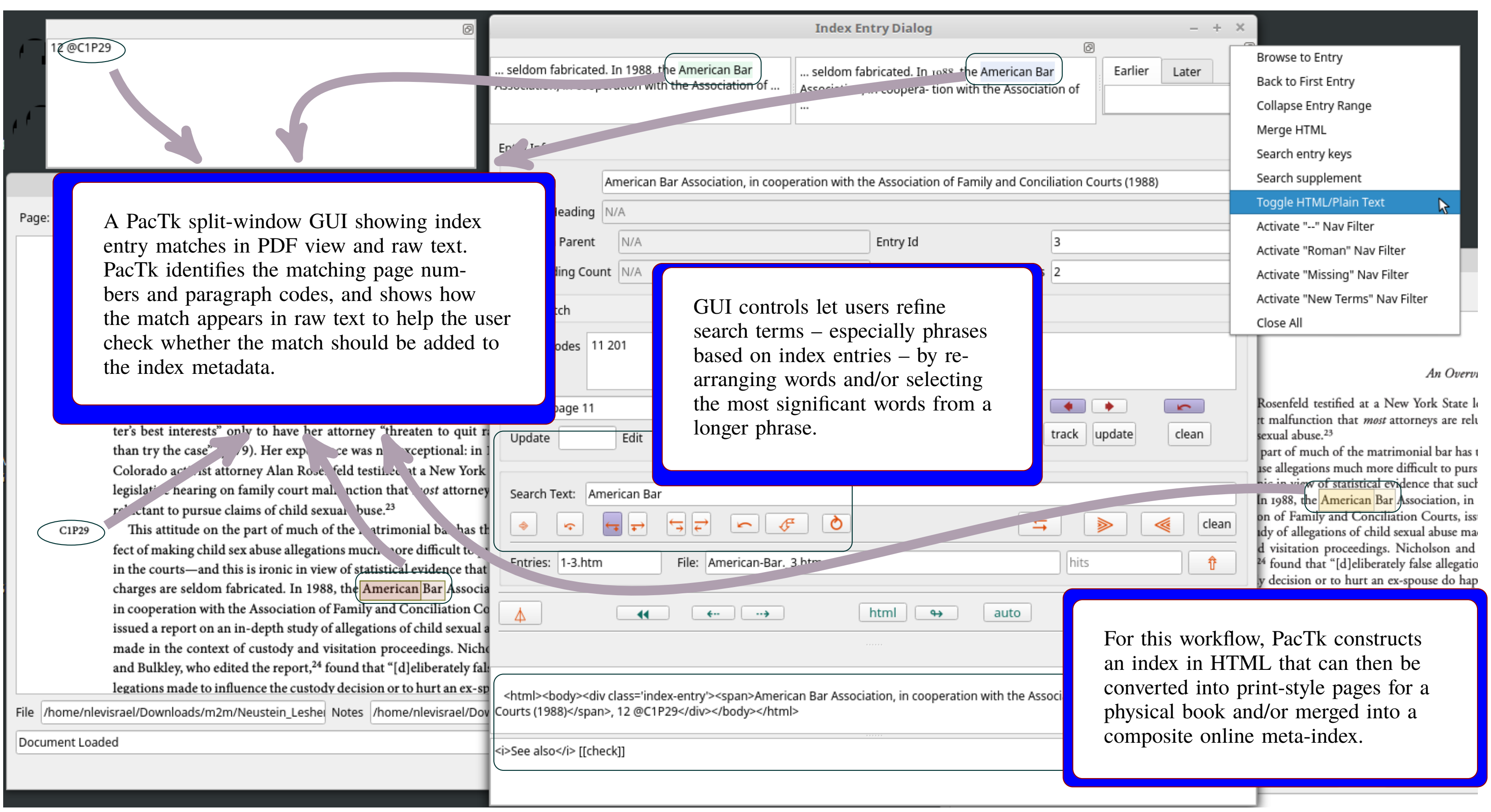


Figure 3: GUI functionality for compiling and curating indexes

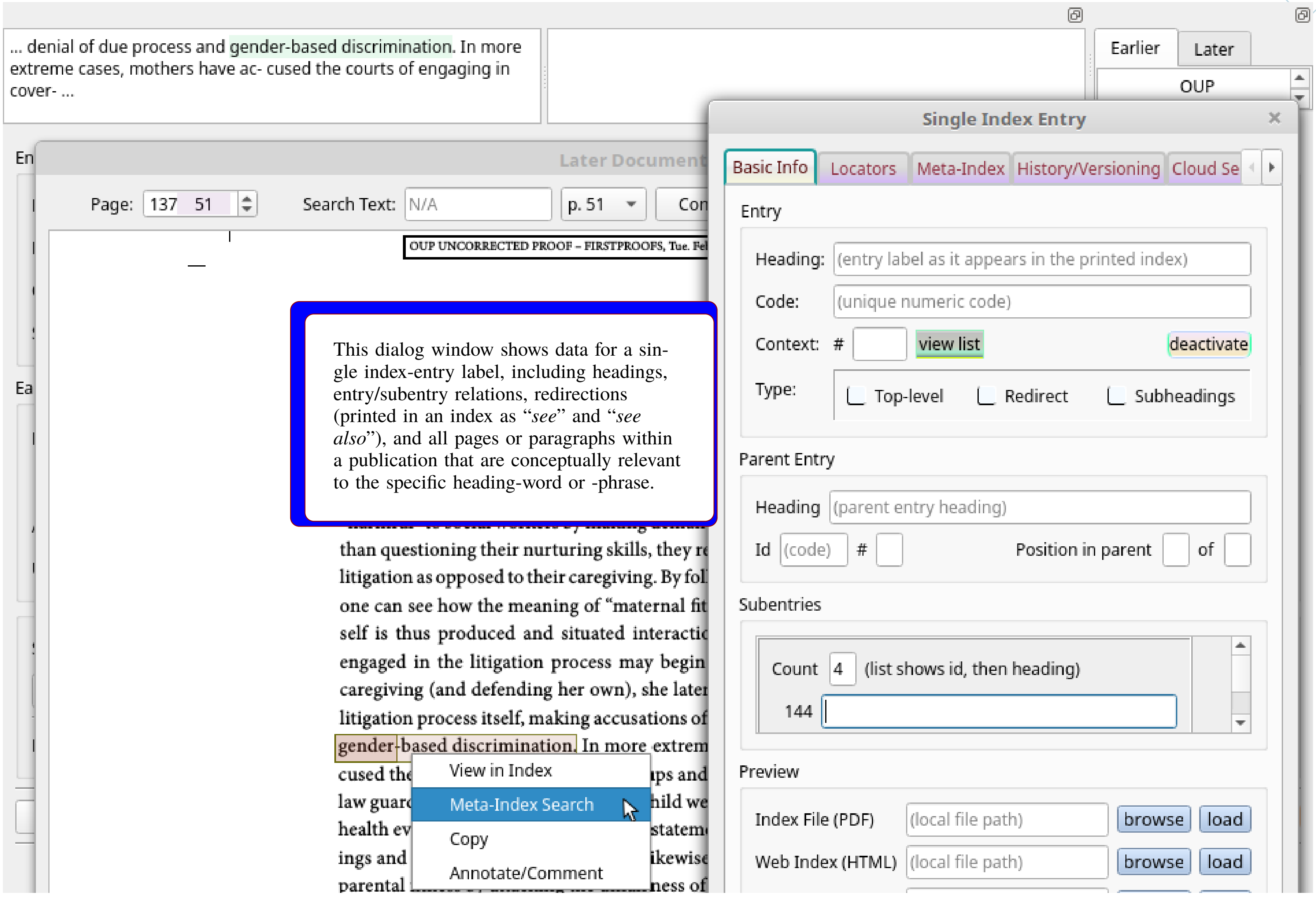


Figure 4: An index entry dialog window (and context menu for single and meta-index views)

Separating Data from Presentation Different publishers or book series may choose different visual styles for their indexes. Representing indexes behind the scenes via structured data objects allows them to be curated in abstraction from the specific layout chosen for final publication.

PDF Context Menus Authors/editors may prefer to add new index entries or locators directly from PDF pages. PacTk-compliant viewers will read metadata that specifies which paragraphs are in scope at different page-coordinate regions, so paragraph ids for any location can be ascertained from mouse events (e.g., the dropdown point of a context menu).

Index-Building GUI Components Index entries are multi-faceted objects that naturally lend themselves to dedicated front-end displays, such as dialog windows (Figures 3 and 4). For example, entries typically include a heading label and then one or more locators (e.g., paragraph ids, replaced eventually with page numbers), then possible subheadings with their own locators, plus "redirectors" (*see* or *see also*) applied either to overall entries or to subheadings. This represents a lot of information to keep track of for each entry, and indexes can include hundreds of entries. Although indexes are eventually shown to readers as ordinary PDF pages (or similar formats), during document prep it is useful to subdivide indexes into individual GUI windows for each entry, with options to navigate between adjacent entries or to traverse on larger scales (e.g., one chapter to another), plus to create new entries on the fly.

Index-Guided Text Searching Indexes are curated lists of headings and locators selected by authors/editors for relevance. Whenever a search is posed against a manuscript in any context — e.g., in the query bar of a PDF viewer, or an online search engine, or XQuery-style terms evaluated vis-à-vis JATS files — index metadata can be consulted just in case search terms overlap with headings already included in an index. Such functionality may be augmented by extending index metadata to include concepts related to index entries even if those secondary concepts are not indexed themselves (as with semantic clusters, mentioned above). In effect, index metadata may declare that all locators modeled as relevant for their specific entry are also relevant to some list of associated concepts, such that whenever those latter concepts are targeted in a query the explicit locators are included among the query results.

Series or Archival Meta-Indexes Once a standard format is developed for indexes in some document collection, it becomes possible to pool multiple instances into a common archive. Each time a new publication is finalized, all of its index terms could be inserted into a bibliographic database allowing users to search across multiple books/manuscripts in a given book series, journal collection, or document repository. Such meta-indexes can support searches that are more precise and granular than conventional search-engine technologies based on occurrence vectors and stemming/lemmatization alone (Figure 5).

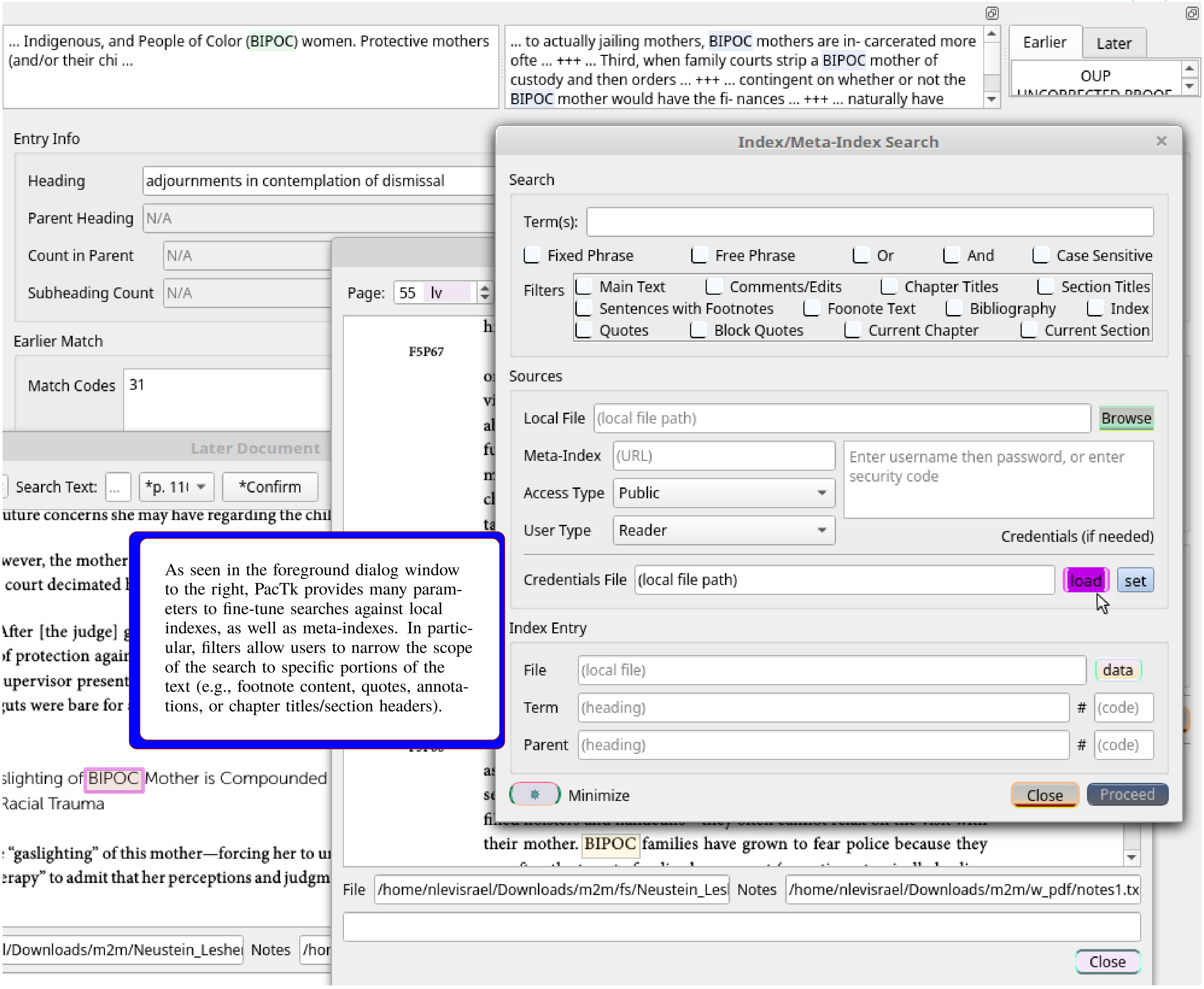


Figure 5: Dialog window for local and meta-index searches

Academic Search Engines In online contexts, "indexing" typically refers not to human-visible book indexes but rather to compressed data files that catalog nontrivial words and concepts in a manuscript. Such indexing is typically non-contextual; for example, it is not represented whether a word/phrase is present in text with a special purpose (quotes, footnotes, chapter/section titles, etc.) and sentiment/discursive roles are ignored (e.g., a document might be identified as *about* some topic, but more specific criteria such as arguing *for*, *against*, or neutrally *evaluating* theories or claims are not modeled). Granularity may be limited by the scope of an archive, because efficient queries can only expend a few seconds per document if searches are to be conducted against a large collection.

Nevertheless, search functionality can take steps to increase query specificity via meta-indexes and related metadata. For one thing, users may specifically consult search areas that are restricted in scope — say, a single University Press, book series, repository (PubMed Central, for example), or journal collection. Publishers who curate meta-indexes for mid-size collections can support much more detailed evaluations per document; a book series might have several dozen titles, not tens of thousands. **PacTk** supports construction of corpora query languages that express granular searches over multiple documents, with similar logic to filtered/contextualized text searches in a single **PDF** document (Figures 6 and 9). Even where queries are executed via conventional vector-database methods, **PacTk** allows authors or digital editors to exercise control over how word-occurrence lists are compiled. For example, **TSOM** objects can include lemmatized word-lists per sentence; authors may then control for circumstances such as two different meanings reduced to the same linguistic stem, which might not be properly ingested by search engines indexing documents through their default algorithms.

Although **PacTk**'s originating focus is publishing — where the primary components of a repository are manuscripts/documents in formats like **PDF** — this technology could potentially be used in other environments. For instance, in clinical trials and point-of-care software the fundamental assets are Electronic Health Records; but search features, as well as plugins and multimedia (discussed next), might be extended to include resources such as diagnostic images, cytometry gating, assay graphics, tumorogenesis simulations, etc. Pulishing-related plugin architecture (e.g., document viewers in scientific software) may be adapted and resued for these **EHR** contexts.

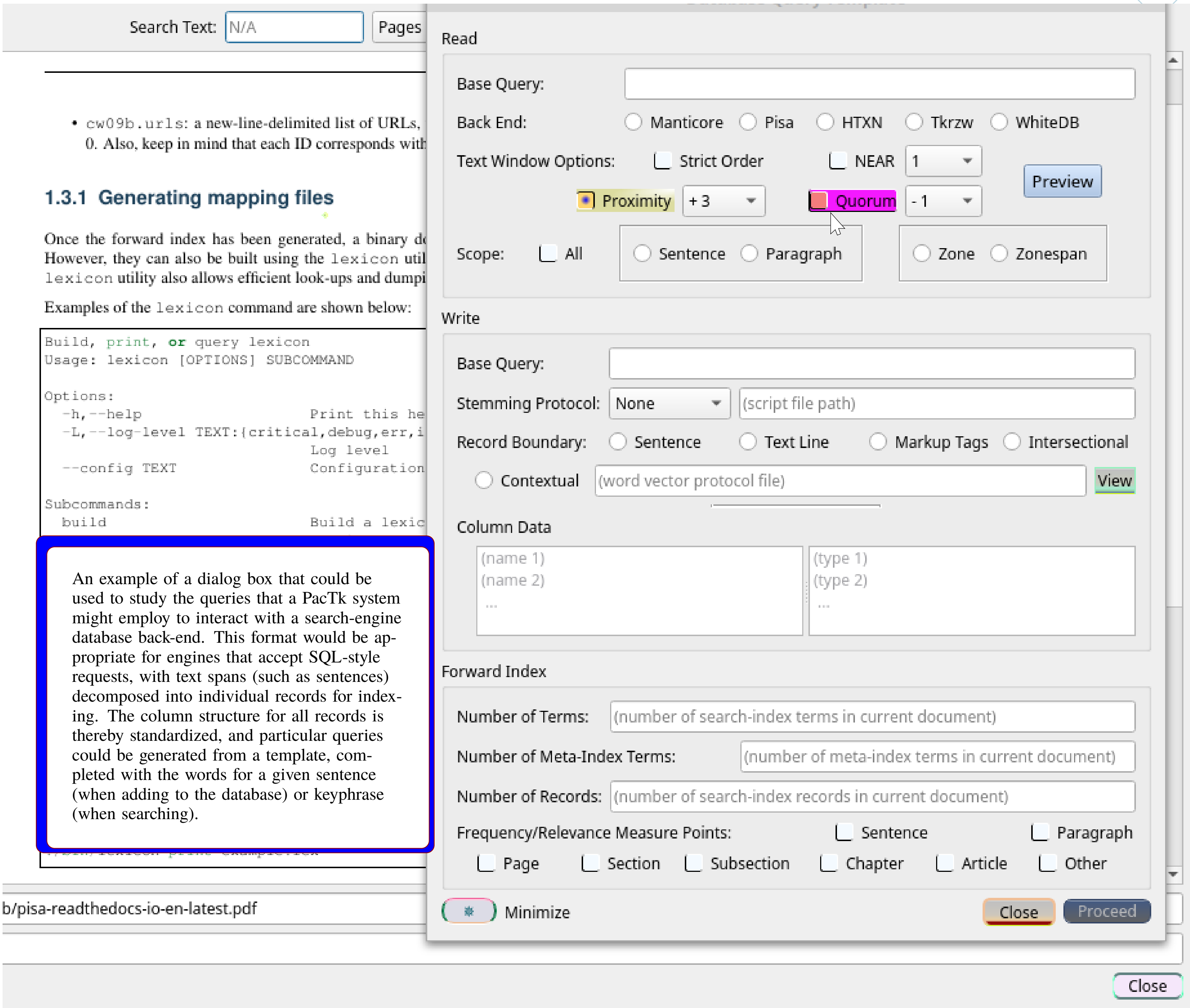


Figure 6: Configuring default back-end queries

Accurate Search Portals For users accessing search functionality through online **HTML** portals, extra programming is necessary to properly show results derived from **PDF** documents. Figure 12 (last page) shows two excerpts taken from publications included among one publisher's lists of subconcepts under the "engineering" heading. Both examples show irregular line spacing and improper rendering of text with distinct font-styles. It is clear that the intermediate files, from which **PDF** was produced as expected, were less successful in the context of **HTML** generation (an accurate **PDF** is displayed at the graphics bottom). These pages show a weakness in **JATS**-style encodings that can be alleviated by a sentence-object model where styling objects have both LaTeX and **HTML** output content.

Another source of transcription errors is character strings which are not ordinary language and therefore require special encoding rules. The book on the left in the Figure 12[1] was a survey of biomedical software engineering in several domains, including text mining, radiomics, and immuno-oncology. Vis-à-vis text mining, the book reviewed **CORD-19**, a large open-access document repository curated by the Allen Institute for **AI** (**AI2**) to spearhead Coronavirus research. This corpus, unfortunately (and as acknowledged by **AI2**) had inconsistent (**PDF**-based) text extraction; e.g., in the aforementioned analysis the authors gave examples of a chemical formula that was printed in different styles in distinct publications reproduced within **CORD-19**, which could cause the overlap on that common semantic element to go unrecognized by text mining algorithms. In response to problems both with special-purpose and ordinary text representation, **AI2** issued a "call to arms" encouraging publishers to adopt standardized machine-readable encodings as supplements to their **PDF** (and **HTML**) outputs: "Existing schemas like [**JATS**] can be too coarse-grained to capture all ... metadata elements, [and] representations ... vary greatly across publishers who use them. To improve metadata coherence across sources, the community must define and agree upon an appropriate standard of representation"[2] **PacTk** is an example of technology that could satisfy these goals.

---

[1] Amy Neustein and Nathaniel Christen, *Innovative Data Integration and Conceptual Space Modeling for COVID, Cancer, and Cardiac Care* (Elsevier, 2022).

[2] Lucy Lu Wang, *et al.*, "CORD-19: The COVID-19 Open Research Dataset," Allen Institute for AI, 2020. https://aclanthology.org/2020.nlpcovid19-acl.1.pdf

Granular Locators Current indexing tools are migrating to paragraph ids (rather than page numbers) for index preparation, although numeric locators still appear in the human-visible digital (and, of course, printed) versions. For best accuracy, it is worthwhile to partition paragraphs that contain block quotes, enumerations, or other subdivisions and assign unique ids to each part (see Figure 7). For example, segments of text before and after a block quote (plus the quote itself) can have distinct ids, even if they are part of the same larger, encompassing paragraph.

Given this precision, digital indexes even in PDF documents can benefit from paragraph ids. The right-hand part of Figure 7 shows an index with color-coding to indicate such details as subdivisions (e.g., block quotes) and paragraphs split between two pages. In the depicted document, all locators are clickable and scroll to a precise location (top of paragraph or of page).

Bidirectional Indexes and Text Extraction Assuming index entries are pooled into an archival data structure, each entry is a potential cross-reference point between multiple documents. The concept of a “locator” may thereby extend beyond a single publication; from any one digital index users could initiate searches for matching entries in other books or articles.

To support such functionality, text encoding should describe structured objects (such as **date** and **string-name** tags in JATS) wherever segments follow canonical encodings beyond just natural language (e.g., citations, references, bibliographic entries, and quoting/transcription conventions). Also, via **PacTk** meta-indexes can enforce consistent encoding for special characters (e.g., quote marks) and structured segments (e.g., proper-name initials) in cases where different style guides have inconsistent recommendations (for example, whether or not to include spaces between consecutive initials). Treating such portions of a document as unstructured text content can obscure cross-references and potential micro-citations, as well as cause errors within manuscripts internally.

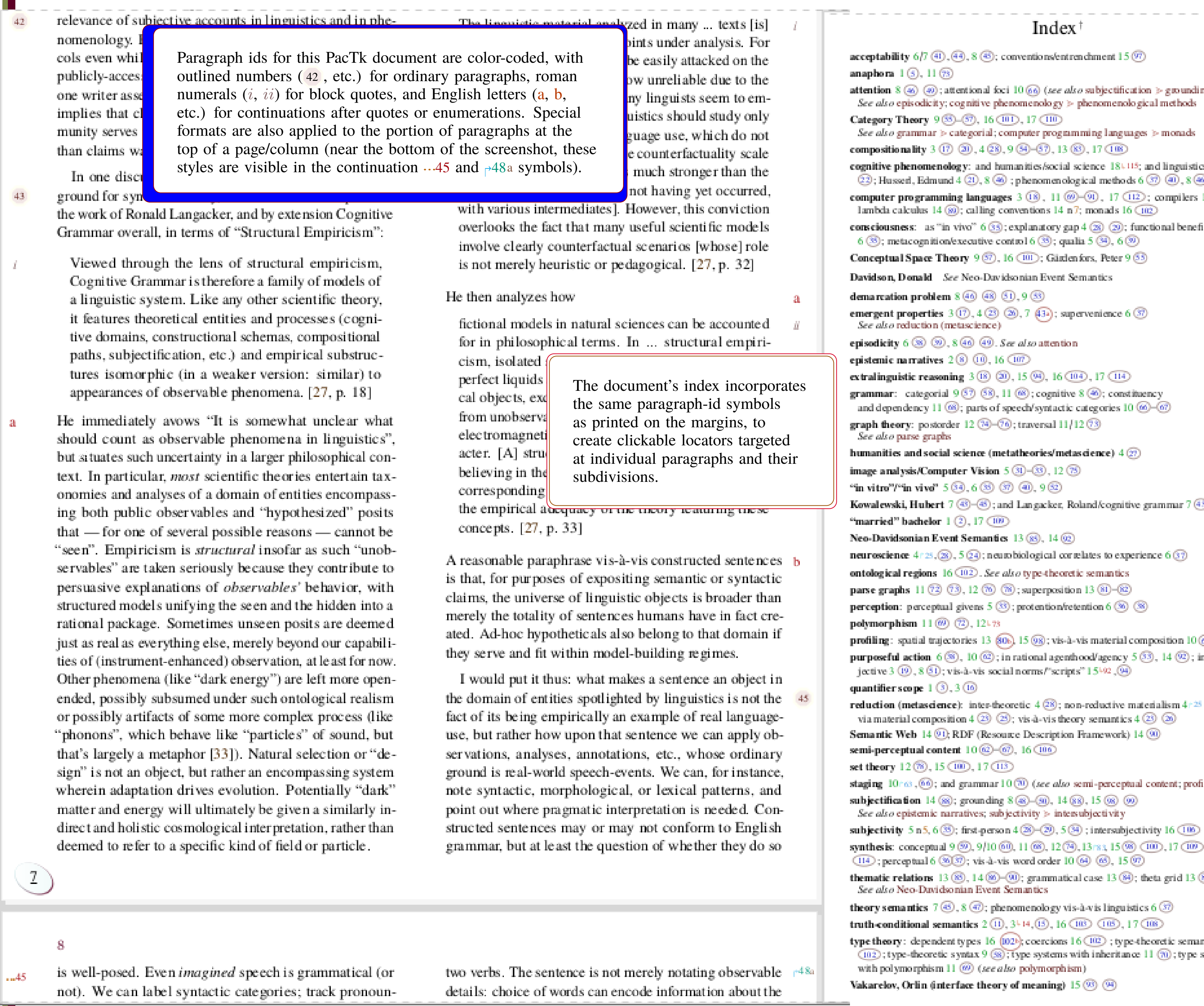


Figure 7: Subdivision Formats for Paragraph IDs

Indexing Digital/Multimedia Repositories For some initiatives, authors deposit content in a variety of media, including but not limited to text documents. For example, the Cancer Biomedical Informatics Grid (caBIG) project is an oncology research network across multiple continents; institutions can join the network if they adhere to "Vocabularies and Common Data Elements" (VCDE) standards applying to APIs, metadata, provenance declarations, execution models, and so on, to support cross-network interoperability (for example, data from one institution could be plugged in to simulations from another). Preparing indexes for publications deposited to a structured research network such as caBIG (now succeeded by the National Cancer Informatics Project) optimally includes aligning each index with semantic protocols. For example, index entries can be linked to VCDE terms wherever possible. Likewise, text- and index-search APIs can be subsumed under the research grid API policies; the shared versioning system can be applied to iterations of a manuscript; and and metadata can cross-reference text publications with other submissions by the same (or affiliated) authors.

Similarly, according to standards such as FAIRdata, publishing findable, accessible, interoperable, and reusable data sets goes beyong just making raw data available on an open-access platform. In addition, FAIRdata defines documentation and metadata protocols, so that each statistical parameter (for example) is described with range, measurement, distribution, and similar analytic information.

On a scientific grid such as caBIG, many assets are executable code for "*in silico*" simulations. From a user's perspective, however, such executables often function as multimedia resources, producing two- or three-dimensional graphics that are pedagagoically analogous to image series or videos. Such computer code thereby connects to publications with a similar rationale as videos, digital maps, and other highly-visual content types. For instance, analysis of habitat regrowth after a forest fire can be based on footage of actual locations or of agent-oriented simulations that provide mathematical models for stages of regeneration (which plant and animal species become reestablished at which point in time). Indexing and cross-referencing policies that are adapted for image and video material can thereby be extended to code libraries as well. Controlled vocabularies and other semantic standards apply to the familiar elements of computer code — procedures, classes, data types, inheritance/containment hierarchies, pre- and post-conditions, etc. — but in a "research grid" context graphical displays often function as assets in their own right and can be annotated and indexed as such. A 3D simulation, for instance, functions essentially as a video whose specific frames evolve algorithmically as a product of parameters and initial conditions that may be modified for each run (indeed, videos could be taken directly from dynamically evolving GUI windows).

Disciplines such as systems biology, environmental science, quantum cosmology, and neurophysics are characterized by research networks that explore complex systems from multiple angles, and leverage resources combining data, code, graphics, simulations, and natural-language text. PacTk can help authors expand indexing protocols to encompass this wider range of scientific content.

In particular, we propose a "FAIRdata Systems Biology Dialect" (FSBD) to standardize C++ code annotations according to FAIRdata principles. While not restricted to systems biology, this protocol would reflect coding requirements for architectures such as caBIG, with the goal of a unified meta-index paradigm encompassing data, code, and visual assets on decentralized scientific research grids. In addition to metadata, FSBD would address concerns specific to modular software engineering, such as merging component-local changes to a common database; discovery, validation, and integration of new modules; and reading intracomponent annotations to create shared, searchable documentation (form fields, user-initiated operations, authentication, component capabilities, etc.).

For example, Lynch (1997) studied "the judge (in criminal court) as a figure within the collaborative production of a courtroom hearing . . ." (p. 99). Lynch followed Garfinkel, who "transforms Durkheim's (1938) classic aphorism, 'the objective reality of social facts is sociology's fundamental phenomenon,' by setting out an ethnomethodological alternative to Durkheim's analytic sociology . . . [which is] to examine the regularities . . . [of social life] . . . and to explicate how social order is achieved reflexively in and through the unrelenting practices of local parties who dwell in a society" (p. 100). In keeping with the ethnomethodological tradition, Lynch did not "objectify" judges' work by imposing social-psychological interpretations on their social actions—examining gender or race, or trying to go "behind the scenes [to uncover what was] 'in the judge's head,' behind the closed doors of the judge's chambers, or in the judge's social background" (p. 99). Instead, he studied how judges were "publicly available in the embodied production of the hearing" (p. 99) and examined closely how they displayed an orderly, publicly observable (by interactants in the courtroom setting)[3] accounting of "their judicial actions and reasons as constituents of the courtroom hearings" (p. 125). Like Lynch, we, in our study of the family courts, do not treat judicial decision making as "a composite of variables in a regression analysis [that is] specified by measures of socio-economic status, gender, educational attainment, ethnicity, regional background, or any other social factor or combination of factors" (p. 100). If we were to do

Figure 8: Index Search in a PDF Front-End Context

Rethinking the relationship between pure and applied research, between universities and communities, means a changing landscape for publishers as well. Although all biomedical research (and scientific work in general) is motivated in principle by practical benefits, translational emphases promote the institutional and informatics foundations for "bench to bedside" translation to happen as quickly and comprehensively as possible. Scholars in the field of "translational science," for example, have developed a five-part model of the evolution of practical interventions,[3] of which the fourth involves commonplace adoption of (once-)novel practices in clinical settings and the fifth marks concerted efforts to adopt useful interventions in a wide variety of settings (e.g., not restricted to protocols available only in a few hospitals). Such models suggest roadmaps to help guide deployment once research progresses through multiple stages of concrete implementation. Published materials (not necessarily restricted to journal articles) may reflect data and observations at each of these stages.

Apart from theoretical models along these lines, the key details of translational deployment are often institutional and socioeconomic in nature. Sustaining a research project long enough to rigorously assess empirical outcomes is a different process than procuring funding, labs, and teams to work through the initial research goals on purely scientific grounds. In the *Data Integration* book mentioned previously, the authors identified numerous innovative software initiatives — e.g., the caBIG Digital Model Repository (DMR) mentioned on the previous page — that were discontinued despite promising provisional results. Other frameworks, such as the University of Pennsylvania's Cancer Phenomics Toolkit, evince novel biocomputational ideas but have not been scaled up or replicated beyond their academic point of origin. These suggest fortuitous connections between technical and translational priorities. Beyond originary research, the impetus for further development and adoption of promising software, interventions, or clinical protocols can be supplied by social responsibility and a desire to make effective health care options (for example) available to a broad spectrum of the community. Research initially funded and supported on purely technical grounds might then be able to further develop and expand by emphasizing social/translational benefits.

From a publishing standpoint, translational science implies that a research infrastructure should sustain and evolve over multiple phases of a scientific project, which extends beyond the time-frame of individual books or articles. This is reflected in part by protocols such as "Research Objects" where supplemental materials (graphics, data sets, utility computer code, and so on) are presented in standardized ways. It is also manifest in notions of replication and data transparency: models such as Minimum Information Standards that illustrate experimental/data-acquisition modalities to facilitate subsequent investigations confirming or extending the scope of published work.

More broadly, translational priorities imply that variegated stakeholders — not only scientists but also practitioners, funders, government representatives, etc. — are best served by integrated packages describing the scope, goals, and (possibly intermediate) results of research projects. In current practice, multiple files and resources associated with academic publications are often scattered in different places, without clear provenance or interoperation. Specifications such as the Research Object protocol only partially alleviate this problem.

Against this background, **PacTk** proposes a "Translational Research Object Protocol" that honors the goals of both Research Objects (as a paradigm for dissemination of scientific works) and Translational Science (as a collation of best practices for emphasizing social benefits of research innovations). According to this protocol, individual publications exist in the context of "Project Archives" that have singular entry points both online (potentially integrated with publishers' search portals) and through downloadable resources/data sets. Materials associated with a publication, aggregated through this entry point, would summarize translational possibilities for the documented research and provide a foundation for institutional deployment(s). Individual scientists or students could also utilize Project Archives as encapsulations introducing their interests, background, and expertise to fellow researchers as well as those whose responsibilities include assessing the merits and adaptation potential of projects originating in academic (as well as private/independent) environments.

▷ Data Publishing and Multimedia

Contemporary data transparency and FAIRsharing (Findable, Accessible, Interoperable, Reusable) standards introduce new challenges for publishers. In this environment, published documents such as PDFs serve as only one part of a package of files, alongside research data sets and, often, numerous forms of multimedia content (potentially including audio, video, image series, 3D graphics, and statistical/data-visualization tools and diagrams, plus domain-specific interactive formats). If desired, PDF viewers could themselves be implemented as one portion of a multimedia suite, with functional motifs duplicated across the various components (Figures 10 and 11).

As a "publishing accelerator" toolkit, **PacTk** has several components to help publishers construct these supplemental materials, including cross-referencing text documents with associated data/media assets, and ensuring FAIR usability. Here are several components under development:

Deserialization and Dataset Tools Individual scientific disciplines are often associated with domain-specific data publishing formats that are paired with dedicated deserialization tools (i.e., computer code that reads raw data files into live memory, without relying on special-purpose software). Such code libraries are often maintained by individual institutions or organizations — which might be academic departments, research labs, or independent groups — on behalf of a broader research community. Examples include (among many others) FASTQ and VCF for genomics; HDF5 and ROOT for physics; FITS and netCDF in astronomy and earth sciences;

[3] Also called the "translational continuum"; see for instance https://www.iths.org/investigators/definitions/translational-research

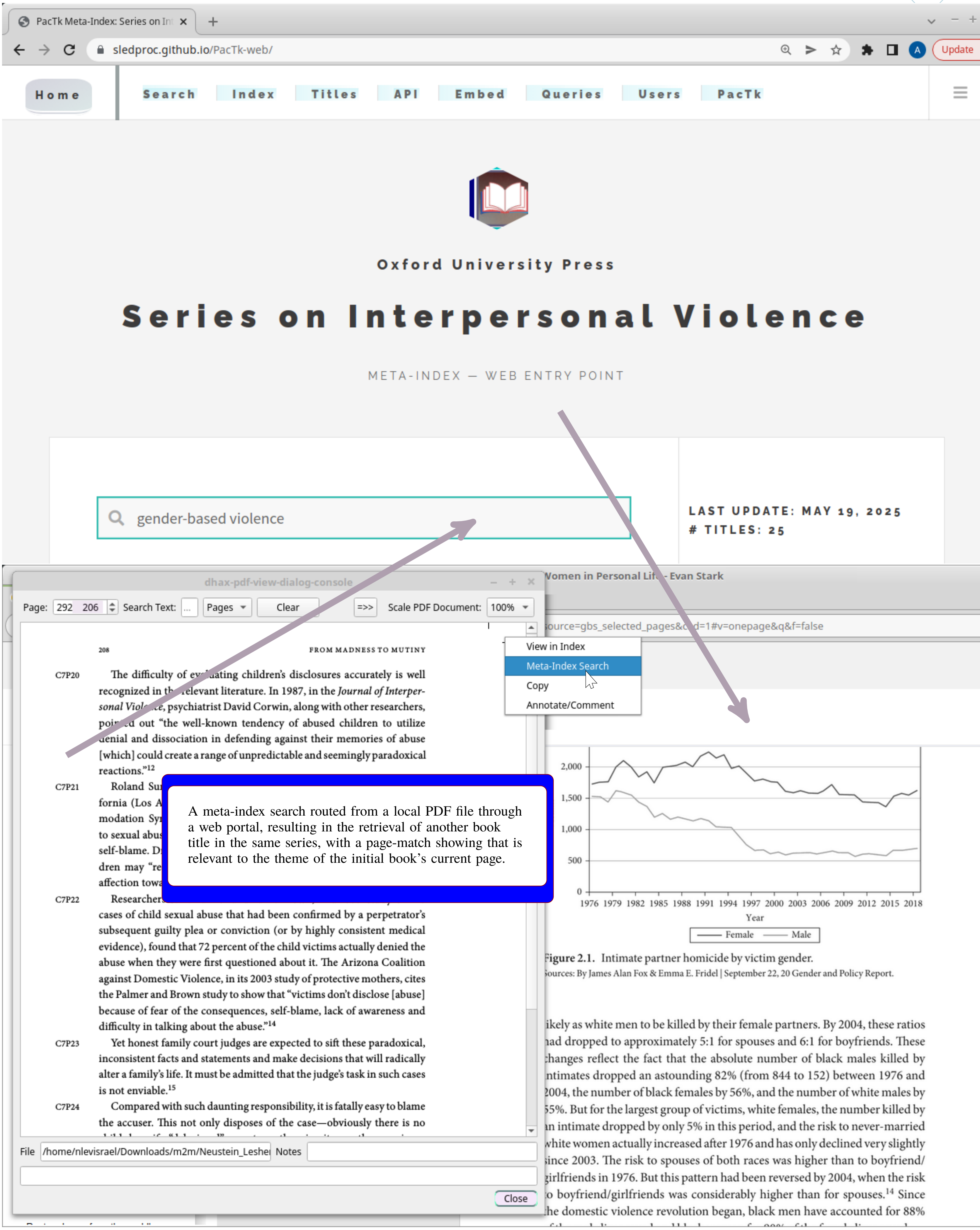


Figure 9: Meta-index search through a web portal

PDB and **MOL** in biochemistry; **IFC** and **BIM** for architecture/engineering; **SEGY** and **BAG** for geophysics and oceanography; **CoNLL-U** in computational linguistics; **DICOM** and **OMETIFF** for bioimaging; and **GIS**-indexed attribute layers and shapefiles for geospatial data sets.

Some domain-specific file formats are derived from general-purpose specifications such as XML, CSV, and JSON. Others employ idiosyncratic text and/or binary serialization protocols, so that custom parsers are necessary so as to consume byte or character streams from relevant raw data files. Even when the actual encoding is performed via a generic standard like XML, dedicated deserialization code will still be needed — ordinary parsers may convert a given XML document to a DOM infoset, but extraction algorithms still must navigate the document hierarchy and obtain text strings from element nodes, which become the input for initializing object data fields.

PacTk seeks to address these issues by offering a platform-independent Virtual Machine that is easy to embed in scientific applications or build as a standalone program. Scripting or query language can then emit code for this specific VM to take responsibility for deserializing encoded data sets. Depending on format, VM instructions might navigate through graph-like structures in a standardized profile (like XML) or implement and/or integrate with parsers for custom input file types, via postprocessors or rule-match callbacks.

Via these technologies, publishers may ensure that data sets are accessible so long as there are deserialization libraries targeting a PacTk VM, rather than duplicating effort (requiring multiple libraries for different operating systems and programming environments).

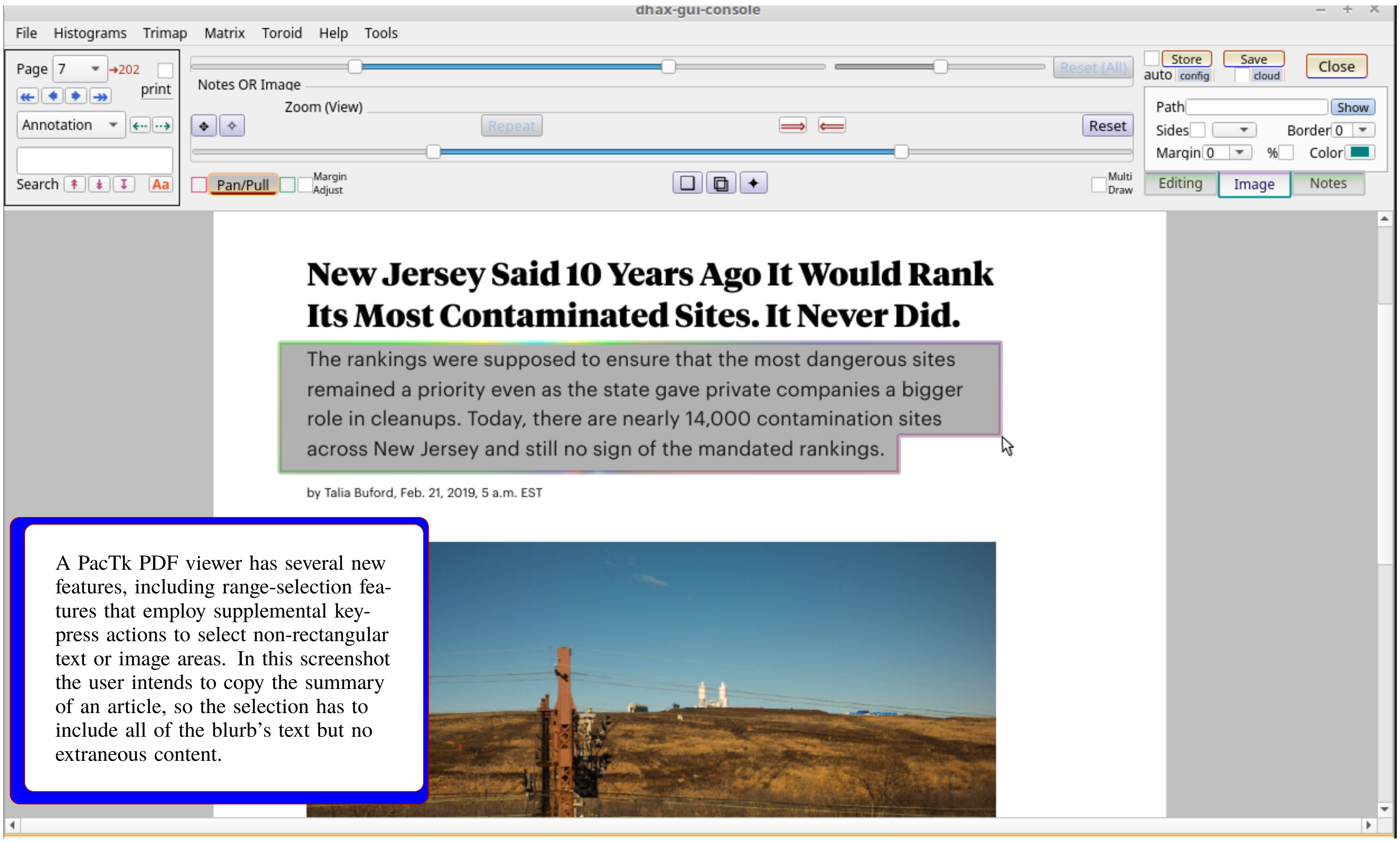


Figure 10: Flexible range-select features

Analytic and Demonstrative Code

Many data sets are published alongside copies or samples of code used to analyze the raw data. Code sharing is important for transparency (allowing outsiders to double-check the accuracy of calculations and conclusions drawn therefrom) and reuse (replication of experiments/research projects). Data publications (bundled as Research Objects or similar scientific packages) may also include implementations for plugins targeted at applications charged with loading the raw data files. PacTk's object system includes several classes encapsulating plugin functionality (such as dynamic-library symbol resolution) and components exposed by host applications to extension modules (main-window taskbars, top/context menus, filesystem access, etc.).

Although such accompanying code might be implemented in many different languages and environments, the VM principles identified above for deserialization tasks also apply to code specific to research data sets. Here, too, accessibility implies that dataset code should be available in a cross-platform manner. Moreover, implementations should proceed via conventions suitable for GUI wrappers and interop, because research methods will often depend on domain-specific scientific or academic software. Ideally, data sets will be examined via plugins specific to applications familiar to the relevant research community, or at least custom-built software that can interoperate with those domain-specific technologies. Unfortunately, many of the most popular scripting formats in scientific/research contexts (e.g., Python, JavaScript, and R) have noticeable limitations vis-à-vis native-GUI integration.

PacTk enables a different approach, wherein languages can be customized to individual data sets and target pure-C++ VMs whose entire parsing, IR, and runtime stack is embeddable (as source files) in C++ GUI applications with no outside dependencies. Compilers and runtime engines for dataset code may thereby be released as part of the publication package itself. The VM and peer components could also be built directly into PDF viewers, multimedia applications (for videos, digital maps, etc.), and scientific computing platforms.

Another **PacTk** benefit for dataset code involves documentation and pedagogical features. In these contexts, code serves not merely to derive specific computations but also to demonstrate the acquisition methods and analytic protocols relevant for a research project. Such details as units of measurement, expected ranges, typed coordinate systems (e.g., ensuring that procedures expecting horizontal-then-vertical coordinate pairs are never called with the converse), compile-time/statically-analyzable pre- and post-conditions, and so forth, help dataset code to serve as a resource clarifying scientific principles and theoretical models that drive published findings. The **PacTk** Virtual Machine includes overload-by-contract, dependent typing (a version thereof compatible with template metaprogramming, as such with some technical differences from, say, IDRIS), native interop (with statically compiled as well as dynamically loaded libraries) and "user-defined" calling-convention features that differ in some technical details from typical language interpreters, resulting in a scripting environment specifically built for open-access data sharing and the deployment of data packages as pedagogical tools.

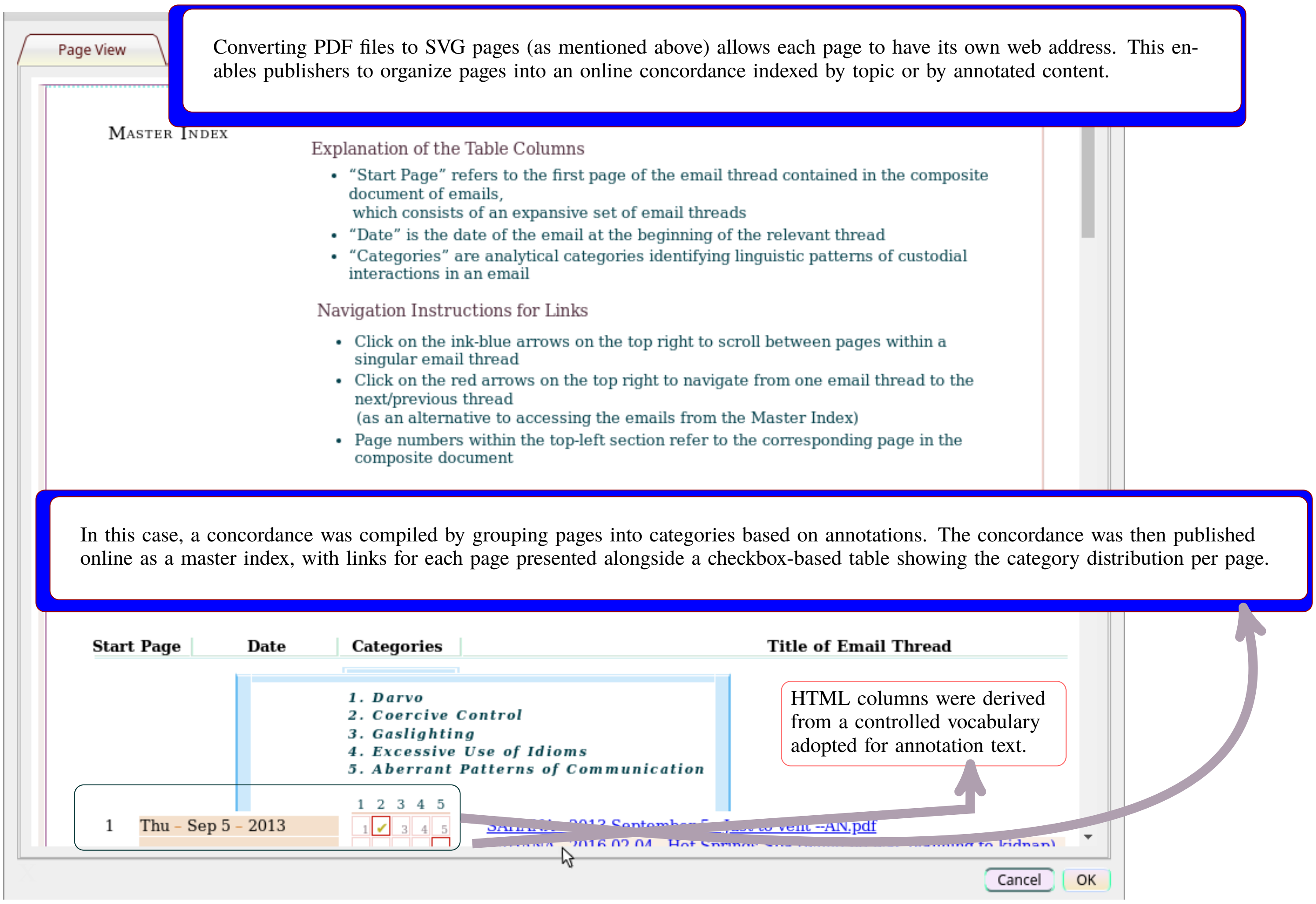


Figure 11: A category-based web-accessible index table for SVG pages

Full-Text Query Languages As discussed earlier in this document, **PacTk** can be utilized as a code library by calling methods on C++ (specifically, TSOM) objects. At the same time, a **PacTk** Virtual Machine supplies an environment for compiling scripting or query languages. These functionalities could be merged, such that the process of extracting information from a PDF manuscript, meta-indexed repository, or bibliographic database might be achieved by formulating and evaluating part of the code in a specialized query language. The techniques involved would be similar to employing XQUERY or XQSE expressions to work with JATS or BITS files.

PacTk features its own flexible front-end parsing framework (mentioned above relative to **GTagML**) that could be used to program query grammars in an extensible, adaptable manner. By leveraging full regular expression capabilities, callbacks to arbitrary code (instead of generative production rules), and directly-executable grammar files (with no separate compilation steps), the **PacTk** parsers are more flexible than conventional LEX/YACC-style compilers. Meanwhile, the VM runtimes allows query evaluators to recognize basic scripting constructions such as locally-scoped variables and calls to arbitrary boolean-valued procedures as filter parameters.

In totality, such features allow text/bibliographic query systems to be implemented in a self-contained and cross-platform manner, with customizations specific to individual publishers and even to each particular journal or book series.

Multimedia Cross-Referencing When a document (book or article) is explicitly paired with a published data set, information about the supplemental package should be included in index/search metadata. This would allow keyword searches to be extended to raw data fields, as well as data-structure parameters (column labels, source-code annotations, procedure names, etc.). Moreover, sections of text may be isolated as cross-reference targets for data-structure parameters as well as individual objects/records. For example, a specific column or field in tabular data structures can be cross-referenced with the paragraph in article text where that field is discussed (its observation/acquisition methodology, units of measurement, valid range delineation, statistical distribution, and so forth).

Cross-referencing between research papers and data sets is a special case of multimedia cross-referencing, where annotations defined in the context of a specific media type assert semantic or thematic relations to contents of a different modality. Concrete examples of multimedia cross-referencing include geotagging video frames; linking **CAD** digital twins with product specs; annotating Regions of Interest within image series with coded labels associated with published raw data; information within natural/social science cross-disciplinary areas such as medical humanities, translational sciences, socio-agronomics, and civil engineering; embedding tags in 360° displays whose content includes **URL** strings that encode identifiers for dataset objects; developing color- and/or shape-coded attribute layers for **GIS** overlays (so users can browse between digital maps and structured object displays); and constructing numeric tuples from **3D** graphics scenes such that particular camera orientation/location and zoom levels could be recorded as a unit, allowing relevant **3D** content to be repeatedly visualized from an identical perspective (in other words, links are embedded in **PDF** files, web pages, software, or any other context so that users can reconstruct a specific **3D** state, as constituted by camera angle/position and zoom settings).

The challenge when implementing multimedia cross-references is to ensure that viewers for two or more divergent media types may properly interoperate. With correctly interpreted cross-references, application users should be able to navigate from a window dedicated to one media/file to a window rendering a different format, and correctly isolate the portion of a file (not just the overall file) implicated in a cross-reference. For example, geotagging video frames should enable users to pause a video at a particular location and then, via the geotag, switch to a map display centered on the geospatial coordinates visible in the video (consider videos documenting progress in an environmental remediation site; or infrastructural and socioeconomic implications of Zoning and Land Use ordinances). Conversely, a **GIS** attribute layer could represent locations for which video content is available, allowing users to watch the relevant videos starting from the appropriate time-point. Such interoperability is only possible if the components and/or applications responsible for rendering the two content-types share a common annotation and message-passing protocol. In the context of **PacTk**, this may be achieved via **TSOM** "message" objects that encapsulate remote-method requests and parameter serialization via **TAGML**-like structures.

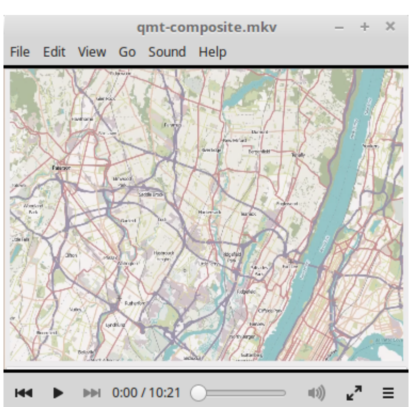
Science and Humanities GUIs Data publishing according to **FAIR** sharing standards often requires numerous project-specific resources in addition to raw data files. Fellow researchers typically need visualization and computational tools to fully leverage published data sets for evaluation and re-use. Unfortunately, relying on generic software to access published information packages is often suboptimal. For example, **CSV** files might be loaded into spreadsheets and/or statistical applications, but simply viewing data files in tabular fashion (even if the information is conducive to such structures) provides only limited value. The same is true for statistical plots and diagrams. Effective evaluation and re-use involves front-end programs that could provide both visual overviews (e.g., via charts, graphs, or histograms) and functionality for examining individual data objects in detail (e.g., via custom-designed **GUI** windows).

Furthermore, many data sets span multiple structural profiles. For example, consider information tracking environmental pollutants, such as published by the Environmental Protection Agency's Toxic Release Inventory (**TRI**) project. Software tools for rigorously studying and analyzing such data could furnish both **GIS** maps (for understanding geospatial details and patterns about where environmental hazards are present) and (bio)chemical software for molecular visualization, or other features to help document public-health and ecological consequences, plus cleanup/remediation protocols for different classes of contaminants. Such components might be further extended by economic data profiling social impacts on surrounding communities, public health information, video and image series. etc.

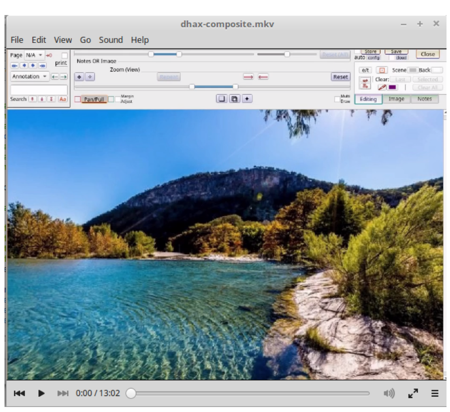
In general, then, publishers should not assume that **FAIR** transparency is achieved simply by sharing raw data files. Instead, open-access materials should in general include plugins or source-code extensions (developed individually for one manuscript; or collectively for a book series, journal title, or document repository) targeting science applications with requisite capabilities to view/analyze domain-specific file types. Similar comments apply to humanities environments, such as front-ends for studying linguistics corpora and/or parse-structures; sociological/economic statistics; or graphics tools for digital humanities. **PacTk** can help in this context because a single **PacTk** plugin could serve as middleware on top of which extensions could operate that are individualized to specific publications/manuscripts. This may be more efficient than building components against applications' native plugin mechanisms directly.

▷ Video Links

For more information, please see our software demo videos, which show software components that could be integrated into dataset applications and **PDF** viewers:



GIS



360° photography

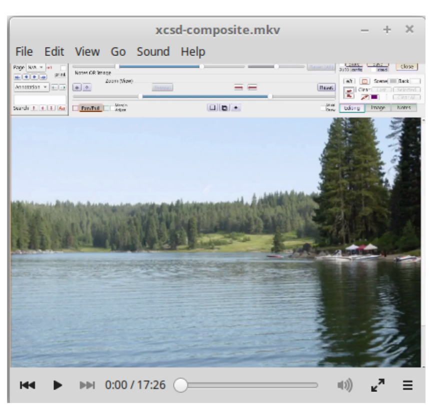


Image Processing

On this page

FASTA

(which stands for Fast-All, a genomics format),

SRA

(sequence read archive, for

DNA

sequencing),

PDB

(Protein Data Bank, representing the

3D

geometry of protein molecules),

MAP

(electron microscopy map),

EPS

(Embedded Postscript), and

CSV

(comma-separated values). There are also tables represented in Microsoft Word or Excel formats. Although these various formats are reasonable for storing raw data, not all of them are actually machine-readable; in particular, the

EPS

, Word, and Excel files need manual processing to use the information they provide in a computational manner. A properly curated data collection would need to unify disparate sources into a common machine-readable representation (such as

XML

).

1

On this page

Definition
Chapters and Articles
Related Terms
Recommended
Publications

a. Enter the text shown in A1. Select A2:A501 (holding down

⇧Shift

while tapping

PgDn

or

↓

will speed things up). Enter =RANDBETWEEN(1,100) and commit with

Ctrl

+

↵

(no

⇧Shift

, this time). Select cell A1 and use

Ctrl

+A to highlight A1:A501 and use the *Copy* command (or

Ctrl

2

Limited Support for Research Data-Mining Even though many papers in CORD-19 are paired with published data sets, there is currently no tool for locating research *data* through CORD-19. For example, the collection of manuscripts available through the Springer Nature portal linked from CORD-19 includes over 30 COVID-19 data sets, but researchers can only discover that these data sets exist by looking for a "supplemental materials" or a "data availability" addendum near the end of each article. These Springer Nature data sets encompass a wide array of file types and formats, including FASTA (which stands for Fast-All, a genomics format), SRA (Sequence Read Archive, for DNA sequencing), PDB (Protein Data Bank, representing the 3D geometry of protein molecules), MAP (Electron Microscopy Map), EPS (Embedded Postscript), CSV (comma-separated values), and tables represented in Microsoft Word and Excel formats. To make this data more readily accessible in the context of CORD-19, it would be appropriate to (1) maintain an index of data sets linked to CORD-19 articles and (2) merge these resources into a common representation (such as XML) wherever possible. This research-data curation can then be treated as a supplement to text-mining operations. In particular, queries against the full-text publications could be evaluated *also* as queries against the relevant set collection of research data sets.

3

Two book excerpts showing inaccurate HTML generation from intermediate files whose primary purpose is to produce PDF documents (1 and 2). The common problem visible in the two examples is that textual or character segments which should be treated as horizontal units (subject to glue-spacing algorithms vis-á-vis words to their left and right) are, incorrectly, isolated on their own line, as if they were "vertical mode" boxes instead. Such errors naturally disrupt the surrounding lines with normal content instead. The proper way to address these issues is to specify project-specific HTML encoding rules for non-standard tag entities and/or sentence-level (or smaller) objects, rather than relying on relying on generic HTML generators. The excerpt at the bottom of the screenshot shows a correct rendering of the left-hand passage (3).

Figure 12: HTML Transcription Errors